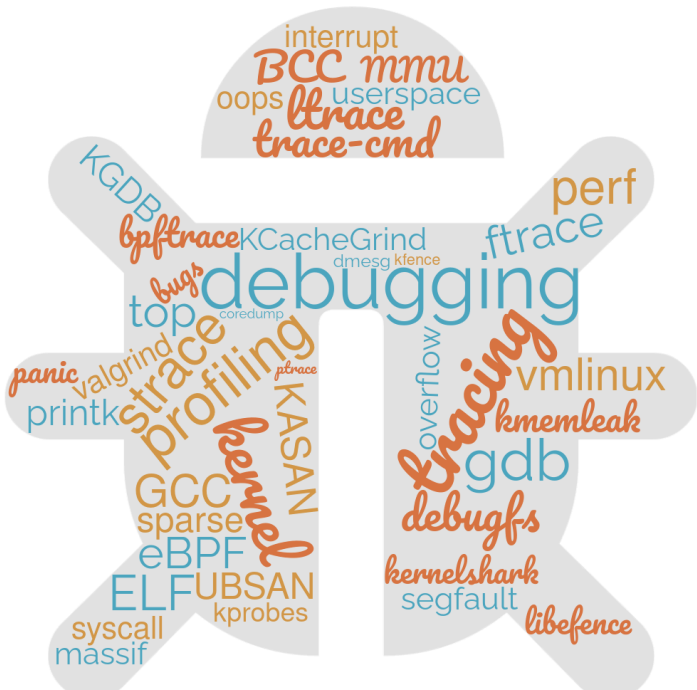
Debugging, Tracing, Profiling



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!

Debugging, Tracing, Profiling

- Debugging, tracing and profiling are often used for development purposes
- All of these methods have different goals which aim at perfecting the software that is being developed
- Requires some knowledge about underlying mechanisms to correctly identify and fix bugs





Debugging

- Finding and fixing bugs that might exist in your software/system
- Use of various tools and methods to achieve that
  - Interactive debugging (With GDB for instance)
  - Postmortem analysis (Using coredump for instance)
  - Control flow analysis (With tracing tools)
  - Testing (Targeted tests)
- Most commonly done through debuggers in development environment
- Generally intrusive, allowing to pause and resume execution

"Everyone knows that debugging is twice as hard as writing a program in the first place. So if you're as clever as you can be when you write it, how will you ever debug it?"
- Brian Kernighan



Tracing

- Following the execution flow of an application to understand the bottlenecks and problems.
- Achieved by instrumenting code either at compile time or runtime.
  - Can be done using specific tracers such as *LTTng*, *trace-cmd*, *SystemTap* etc
- Goes from the user space called functions up to the kernel ones
- Allows to identify functions and values that are used while application executes
- Often works by recording traces during runtime and then visualizing data.
  - Implies a large amount of recorded data since the complete execution trace is recorded
  - Often bigger overhead than profiling.
- Can also be used for debugging purpose since data can be extracted with tracepoints.



Profiling

- Analysis at program runtime to assist performance optimizations
- Often achieved by sampling counters during execution
- Uses specific tools, libraries and operating system features to measure performance.
  - Using *perf*, *OProfile* for instance.
- First step consists in gathering data from program execution
  - Function call count, memory usage, CPU load, cache miss, etc
- Then extracting meaningful information from these data and modify the program to optimize it



Event sources

- Some activities like tracing or profiling involve some preliminary data collection
- Those data are gathered from different **event sources**, which can be of various types.
  - Some low-level events are gathered directly by the hardware (eg: CPU cycles, MMU exceptions...)
  - Some events are generated by some code explicitly added to generate traces, either in an application or in the kernel: those are **static tracepoints**
  - Some are generated by instrumentation added at runtime (ie without having to modify/rebuild the application and/or the kernel): those are **dynamic probes**